

1. Introducción a la codificación

La parte de codificación de la memoria describe la implementación del sistema vitalXAI: cómo se materializó en código ejecutable el diseño especificado en los capítulos 18 a 27. Mientras que el diseño determina la arquitectura, los casos de uso, las clases, las interfaces, la construcción, la preparación de los datos, la verificación y la implantación, esta parte describe las decisiones de implementación concretas que hicieron realidad el sistema: la organización del repositorio, las convenciones de código, la estructura de cada capa tecnológica y los fragmentos de código más representativos de cada módulo. La codificación es, por tanto, la traducción fiel del diseño a un sistema Python servido por FastAPI y apoyado en MySQL, TensorFlow y los servicios del laboratorio MLOps.

La parte se organiza en seis capítulos. Este capítulo introductorio presenta la visión general de la implementación, la estructura del repositorio, las convenciones de código y las herramientas de calidad; los cinco capítulos siguientes describen la codificación de cada capa tecnológica: el backend (capítulo 29), la ejecución asíncrona (capítulo 30), el motor de diagnóstico y de explicabilidad (capítulo 31), el laboratorio de experimentación MLOps (capítulo 32) y el frontend (capítulo 33). Cada capítulo de codificación presenta los componentes implementados, los fragmentos de código representativos con su explicación y la coherencia con el diseño del sistema.

La codificación de vitalXAI siguió la metodología de desarrollo dirigida por pruebas (TDD) declarada en el plan de pruebas del capítulo 17, de modo que cada capacidad funcional se implementó después de escribir la prueba que verifica su comportamiento. Esta metodología dio lugar a una batería de pruebas automatizadas —más de ciento ochenta pruebas distribuidas entre unitarias y de integración— que acompaña a la implementación y que se integra en el flujo de integración continua (Myers, Sandler, & Badgett, 2011). Los capítulos siguientes describen la implementación real del sistema, referenciando los módulos, los ficheros de pruebas y las decisiones técnicas que la sustentan.

1.1. *Visión general de la implementación*

La implementación de vitalXAI se organiza en torno a las capas definidas en el diseño: la capa HTTP, la capa de servicios, la persistencia relacional, la ejecución asíncrona, el motor de inteligencia artificial y la capa de presentación. La capa HTTP agrupa los routers que exponen los endpoints de la API; la capa de servicios concentra la lógica de aplicación —criptografía, sesiones, predicción, explicabilidad, informes, laboratorio y cola—; la persistencia relacional se resuelve con MySQL a través de un pool de conexiones; la ejecución asíncrona se materializa en la cola de trabajos y en el worker; el motor de inteligencia artificial agrupa la carga de los modelos y la generación de las explicaciones; y la capa de presentación se compone de las plantillas Jinja2 y los recursos JavaScript del navegador. Cada capa se implementa en un paquete del repositorio, de modo que la estructura del código refleja la arquitectura del sistema.

El lenguaje de implementación es Python en su versión 3.11, que alberga tanto el servidor web como el motor de aprendizaje profundo. El servidor se construye con FastAPI y se sirve mediante Uvicorn; las plantillas se componen con Jinja2; la persistencia se accede mediante el conector MySQL de Python; el aprendizaje profundo se apoya en TensorFlow y en la librería Transformers de Hugging Face; y la seguridad de las sesiones se resuelve con bcrypt, python-jose y slowapi. Estas tecnologías, junto con el resto de las dependencias del sistema, se describieron en el entorno de construcción del capítulo 24, y su implementación se detalla en los capítulos de codificación de esta parte.

La coherencia entre el diseño y la implementación se mantiene mediante la trazabilidad de los módulos: cada router y cada servicio del código se corresponde con los componentes descritos en el capítulo 18 y con las clases de diseño del capítulo 22, y cada endpoint materializa los casos de uso del capítulo 21. Esta correspondencia permite verificar que la implementación satisface el diseño y que los capítulos de codificación pueden leerse como la materialización concreta de las decisiones técnicas ya documentadas.

1.2. Estructura del repositorio

La estructura del repositorio del proyecto refleja la organización por capas de la implementación. El árbol de directorios de la siguiente ilustración representa la estructura del repositorio de vitalXAI y los ficheros principales de cada paquete, con sus agrupaciones funcionales.

La estructura distingue los ficheros raíz de la aplicación, los paquetes de la implementación y los directorios de artefactos. Los ficheros raíz incluyen el punto de entrada `main.py`, la persistencia `database.py`, las dependencias `requirements.txt`, la configuración de calidad `pyproject.toml`, la configuración de la integración continua en `.github/workflows/` y las variables de entorno. Los paquetes `routers/`, `services/`, `templates/` y `static/` contienen la implementación de cada capa; el directorio `pneumoniacnn-main/` alberga el pipeline de experimentación con sus scripts y los pesos de los modelos; el directorio `tests/` contiene las pruebas unitarias y de integración; el directorio `scripts/` reúne los scripts operativos de arranque y de migración; y los directorios `sql/`, `training_results/` y los subdirectorios de `static/` conservan los artefactos generados y los recursos de la aplicación.

```

vitalXAI/
├── .github/
│   └── workflows/
│       └── ci.yml
├── routers/
│   ├── admin.py
│   ├── auth.py
│   ├── history.py
│   ├── inference.py
│   ├── queue.py
│   └── trainer.py
├── services/
│   ├── auth_service.py
│   ├── chatbot_service.py
│   ├── csrf_middleware.py
│   ├── lang.py
│   ├── ml_engine.py
│   ├── mlops_engine.py
│   ├── pdf_generator.py
│   ├── pdf_generator_mlops.py
│   ├── queue_worker.py
│   ├── rate_limiter.py
│   ├── trainer_engine.py
│   └── xai_generator.py
├── static/
│   ├── js/
│   │   ├── admin.js
│   │   ├── dashboard.js
│   │   ├── i18n.js
│   │   └── training.js
│   ├── uploads/
│   ├── results/
│   └── reports/
├── templates/
│   ├── dashboard.html
│   ├── index.html
│   ├── login.html
│   ├── register.html
│   └── training.html
├── tests/
│   ├── unit/
│   └── integration/
├── pneumoniaccnn-main/
│   ├── code/
│   │   ├── 1_train_kfold.py
│   │   ├── 2_train_transformer_kfold.py
│   │   ├── 3_evaluate_statistics.py
│   │   ├── 4_external_validation.py
│   │   ├── 5_evaluate_delong.py
│   │   ├── 6_xai_qualitative.py
│   │   └── 7_xai_quantitative.py
│   └── results/
├── scripts/
│   ├── demo_start.bat
│   ├── demo_start.ps1
│   ├── migrate_passwords.py
│   └── migrate_roles.py
├── sql/
├── training_results/
├── .env.example
├── .gitignore
├── database.py
├── main.py
├── pyproject.toml
├── README.md
└── requirements.txt

```

Ilustración x - Estructura del repositorio de vitalXAI

1.3. Convenciones de código y decisiones transversales

La codificación de vitalXAI sigue las convenciones de estilo de Python definidas en la guía de estilo PEP 8 (van Rossum, Warsaw, & Coghlan, 2001), aplicadas de forma automática por la herramienta de análisis estático ruff. La configuración del proyecto fija una longitud máxima de línea de 140 caracteres y la selección de un conjunto de reglas que abarca los errores sintácticos, las advertencias, las importaciones, las convenciones de nombres, las actualizaciones de sintaxis, las comprobaciones de seguridad y las mejoras de simplificación. La aplicación de estas reglas se verifica de forma continua en el flujo de integración, de modo que el estilo del código se mantiene consistente en todo el repositorio.

Las decisiones transversales de codificación incluyen el uso de tipos estáticos, la política de comentarios y la gestión de los recursos. El proyecto emplea anotaciones de tipos en los módulos de mayor criticidad —los servicios de autenticación y seguridad—, verificadas con la herramienta mypy, de modo que las funciones documentan sus parámetros y sus valores de retorno y los errores de tipos se detectan en tiempo de verificación. Los comentarios se reservan para explicar la intención y las decisiones no evidentes de cada bloque, sin narrar la mecánica obvia del código, en línea con las prácticas de calidad del proyecto. Las variables de entorno se leen mediante la función de carga del entorno, de modo que la configuración sensible permanece fuera del código y se suministra en tiempo de ejecución. Los identificadores de los módulos y de las funciones siguen las convenciones de nombres de Python, con módulos en minúsculas separados por guiones bajos y funciones con la misma convención.

1.4. Herramientas y calidad del código

La calidad del código de vitalXAI se garantiza mediante un conjunto de herramientas de verificación integradas en el flujo de trabajo. El marco de pruebas pytest ejecuta la batería de pruebas unitarias y de integración, con la extensión pytest-cov para medir la cobertura de los módulos de la aplicación y la extensión pytest-mock para la simulación de los servicios externos (pytest, 2024). La cobertura de los módulos de la aplicación está sujeta a un umbral mínimo del setenta por ciento, de modo que la batería falla si la cobertura desciende por debajo del umbral. El análisis estático del estilo y de las reglas se resuelve con ruff (Ruff, 2024), y la verificación de tipos se resuelve con mypy en los módulos de seguridad configurados (Mypy, 2024).

La verificación se integra en la integración continua mediante el flujo de trabajo del repositorio, que se ejecuta en cada integración de la rama principal y en cada petición de cambios. El flujo instala las dependencias, ejecuta el análisis estático con ruff y ejecuta la batería de pruebas con la medición de cobertura, de modo que cualquier regresión de comportamiento, de estilo o de tipos se detecta de forma inmediata. Esta estrategia, coherente con la definida en la estrategia de verificación del sistema del capítulo 26, garantiza que la codificación se mantiene estable a lo largo del desarrollo y que los capítulos siguientes describen una implementación verificada y mantenible.

2. Codificación del backend

El backend de vitalXAI materializa las capas de la aplicación descritas en el capítulo 24: la capa HTTP, la capa de servicios, la persistencia relacional y los mecanismos de seguridad transversales. Este capítulo describe la codificación de estas capas, presentando los componentes implementados y los fragmentos de código más representativos de cada uno, con la explicación de las decisiones de implementación que los sustentan. La codificación del backend se organiza en cinco apartados: el arranque y la configuración de la aplicación, la capa HTTP de los routers, los servicios de aplicación, la persistencia y el acceso a los datos, y la seguridad transversal.

El backend se construye sobre FastAPI, servido por Uvicorn, y se apoya en MySQL para la persistencia (FastAPI, 2024; Uvicorn, 2024; Oracle, 2024). La implementación sigue el patrón de fachada ligera descrito en el diseño: los routers se ocupan de la recepción y la serialización de las peticiones HTTP, y delegan la lógica en los servicios, que concentran las operaciones de aplicación y acceden a la persistencia. Esta separación, ya declarada en el capítulo 18, se refleja en la estructura de los paquetes routers/ y services/ descrita en el capítulo 28, y permite verificar cada componente de forma aislada mediante las pruebas unitarias del capítulo 17.

2.1. *Arranque y configuración de la aplicación*

El punto de entrada de la aplicación se implementa en el módulo main.py, que compone la aplicación FastAPI, configura los mecanismos de seguridad transversales, monta los recursos estáticos e integra los routers de los subsistemas. El arranque se gestiona mediante el ciclo de vida de la aplicación: al iniciarse, se inicializa la base de datos y se arranca el worker de la cola; al finalizar, el worker queda detenido con la aplicación. El fragmento siguiente muestra la implementación del ciclo de vida y de la composición de la aplicación.

La implementación del arranque refleja las decisiones del diseño. La inicialización de la base de datos se realiza mediante la función `init_db()` de `database.py`, que crea las tablas del esquema si no existen, de modo que el arranque no requiere un script de instalación previo; el fallo de conexión se captura y se informa sin interrumpir el arranque, en coherencia con el entorno de construcción del capítulo 24. El worker de la cola se arranca mediante `start_worker()`, que crea la tarea asíncrona del procesamiento en segundo plano. Los mecanismos de seguridad se añaden como middleware —las cabeceras de seguridad y la protección CSRF— y el limitador de peticiones se registra en el estado de la aplicación con su manejador de excepciones. Los recursos estáticos y el directorio de resultados de entrenamiento se montan como directorios servidos por la aplicación, y los routers de los seis subsistemas se integran mediante la inclusión de sus enrutadores.

```

@asynccontextmanager
async def lifespan(app: FastAPI):
    try:
        init_db()
        print("Base de datos conectada e inicializada correctamente.")
    except Exception:
        print("ATENCIÓN: No se pudo conectar a la base de datos MySQL.")
    start_worker(app)
    print("Worker de cola iniciado.")
    yield

app = FastAPI(title="X-Ray AI Consultant", lifespan=lifespan)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)
app.add_middleware(SecurityHeadersMiddleware)
app.add_middleware(CSRFMiddleware)

app.mount("/static", StaticFiles(directory="static"), name="static")
os.makedirs("training_results", exist_ok=True)
app.mount("/training_results", StaticFiles(directory="training_results"), name="training_results")

app.include_router(admin.router)
app.include_router(auth.router)
app.include_router(history.router)
app.include_router(inference.router)
app.include_router(queue.router)
app.include_router(trainer.router)

```

Código 1 - Arranque y composición de la aplicación (main.py)

2.2. Capa HTTP: los routers

La capa HTTP se implementa en el paquete `routers/`, con un módulo por subsistema: `auth.py` para el acceso y las sesiones, `inference.py` para el diagnóstico, `history.py` para el historial, `queue.py` para la cola de trabajos, `trainer.py` para el laboratorio MLOps y `admin.py` para la administración. Cada router sigue el patrón de fachada ligera del diseño: resuelve la identidad del usuario, valida los datos de entrada, delega en los servicios y devuelve las respuestas HTTP con los códigos diferenciados. El fragmento siguiente muestra la implementación del endpoint de solicitud de diagnóstico del router `inference.py`, que materializa la subida de la radiografía y el encolado del trabajo.

La implementación del endpoint refleja las decisiones del diseño del subsistema de diagnóstico. La identidad se resuelve mediante el mecanismo común de identificación de SD-001, y las validaciones del tipo MIME y del tamaño se realizan antes de escribir el fichero en disco, de modo que una petición inválida no ocupa espacio ni una posición en la cola. La imagen se conserva en el área de cargas con un nombre basado en la fecha, y el trabajo se encola con un payload serializable que solo contiene el modelo, la ruta de la imagen y el idioma. La respuesta devuelve el estado `queued` con la posición, y el mensaje se localiza mediante el servicio de idioma. Este patrón —resolver la identidad, validar, delegar y responder con códigos diferenciados— se repite en los routers de los demás subsistemas, con las variaciones propias de cada flujo.


```

@router.post("/predict")
async def predict(request: Request, file: UploadFile = File(...), model_name: str = Form(...)):
    user_id = get_user_id_from_token(request.cookies.get("access_token"))
    if not user_id:
        return JSONResponse(status_code=401, content={"error": get_text("no_autenticado")})

    if file.content_type not in _ALLOWED_MIME_TYPES:
        return JSONResponse(status_code=400, content={"status": "error", "message": get_text("solo_imagenes")})

    file.file.seek(0, 2)
    file_size = file.file.tell()
    file.file.seek(0)
    if file_size > _MAX_FILE_SIZE:
        return JSONResponse(status_code=400, content={"status": "error", "message": get_text("imagen_muy_grande")})

    timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"{timestamp}_{file.filename}"
    upload_path = os.path.join("static", "uploads", filename)
    with open(upload_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    job_id = _enqueue_job(user_id, "diagnosis", {
        "model_name": model_name, "image_path": upload_path,
        "lang": get_lang_from_cookie(request),
    })
    position = _queue_position(job_id, "diagnosis")
    return JSONResponse(content={
        "status": "queued", "job_id": job_id, "position": position,
        "message": get_text("diagnostico_encolado").format(position=position),
    })

```

Código 2 - Solicitud de diagnóstico (routers/inference.py)

2.3. Servicios de aplicación

La capa de servicios se implementa en el paquete `services/` y concentra la lógica de aplicación del sistema. Los servicios se organizan por responsabilidad: `auth_service.py` gestiona la criptografía y las sesiones, `lang.py` la internacionalización, `ml_engine.py` y `xai_generator.py` la predicción y la explicabilidad, `pdf_generator.py` los informes del diagnóstico, `mlops_engine.py` y `trainer_engine.py` el laboratorio, `queue_worker.py` el procesamiento asíncrono, `chatbot_service.py` el asistente conversacional y `rate_limiter.py` y `csrf_middleware.py` los mecanismos de seguridad. El fragmento siguiente muestra la implementación de la creación y la verificación de los tokens de sesión del servicio de autenticación.

La implementación de los tokens refleja la política de seguridad de sesiones del diseño. El token de acceso se firma con JWT mediante la clave secreta del entorno e incluye el identificador del usuario y la fecha de expiración; el token de refresco se genera como un valor aleatorio con `secrets.token_urlsafe`, y en la base de datos solo se conserva su hash SHA-256 junto con la expiración, de modo que una credencial reutilizable no se almacena en texto plano. La gestión de la rotación y de la detección de robo, descrita en el capítulo 18, se implementa en las funciones de verificación del mismo servicio, que distinguen el token activo, el revocado dentro del periodo de gracia y el uso de una credencial revocada, revocando todos los tokens del usuario en este último caso.

```

def create_access_token(user_id: int) -> str:
    expire = datetime.datetime.now(datetime.UTC) + datetime.timedelta(minutes=JWT_ACCESS_EXPIRE_MINUTES)
    payload = {"sub": str(user_id), "exp": expire}
    return jwt.encode(payload, JWT_SECRET_KEY, algorithm="HS256")

def create_refresh_token(user_id: int) -> str:
    raw_token = secrets.token_urlsafe(48)
    token_hash = _hash_token(raw_token)
    expires_at = datetime.datetime.now(datetime.UTC) + datetime.timedelta(days=JWT_REFRESH_EXPIRE_DAYS)
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO refresh_tokens (user_id, token_hash, expires_at) VALUES (%s, %s, %s)",
        (user_id, token_hash, expires_at),
    )
    conn.commit()
    conn.close()
    return raw_token

```

Código 3 - Creación de los tokens de sesión (services/auth_service.py)

2.4. *Persistencia y acceso a los datos*

La persistencia se implementa en el módulo `database.py`, que gestiona el pool de conexiones a MySQL y la inicialización del esquema. El acceso a los datos se resuelve mediante un pool de conexiones configurado con las variables de entorno, de modo que las peticiones concurrentes reutilizan las conexiones del pool sin abrir una conexión por operación. El fragmento siguiente muestra la configuración del pool y la inicialización del esquema.

La implementación de la persistencia refleja el modelo físico del capítulo 20 y la generación del esquema del capítulo 24. El pool se configura con los parámetros del entorno y se crea perezosamente en la primera conexión; la inicialización del esquema ejecuta las sentencias `CREATE TABLE IF NOT EXISTS` de las cinco tablas del sistema —`users`, `consultations`, `training_jobs`, `job_queue` y `refresh_tokens`— de modo que el arranque crea el esquema sin un script de migración manual. Los routers y los servicios obtienen las conexiones mediante la función común, aplican las operaciones y las cierran tras confirmar la transacción, manteniendo la integridad de los datos.

```

def _get_pool():
    global _pool
    if _pool is None:
        _pool = MySQLConnectionPool(
            pool_name="mypool",
            pool_size=int(os.getenv("DB_POOL_SIZE", "5")),
            host=os.getenv("DB_HOST", "localhost"),
            user=os.getenv("DB_USER", "root"),
            password=os.getenv("DB_PASSWORD", ""),
            database=os.getenv("DB_NAME", "tfg_pneumonia"),
        )
    return _pool

def get_db_connection():
    return _get_pool().get_connection()

def init_db():
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INT AUTO_INCREMENT PRIMARY KEY,
            username VARCHAR(255) NOT NULL UNIQUE,
            password_hash VARCHAR(255) NOT NULL,
            first_name VARCHAR(255) NOT NULL,
            last_name VARCHAR(255) NOT NULL,
            role VARCHAR(255) NOT NULL
        )
    """)
    conn.commit()
    conn.close()

```

Código 4 - Pool de conexiones e inicialización del esquema (database.py)

2.5. *Seguridad transversal*

La seguridad transversal del backend se implementa en los servicios `csrf_middleware.py` y `rate_limiter.py`, que se aplican a toda la aplicación mediante el middleware y el limitador configurados en el arranque. La protección CSRF se implementa como un middleware que establece una cookie de token en las peticiones seguras y exige su correspondencia en las peticiones que modifican el estado; las cabeceras de seguridad se añaden a todas las respuestas. El fragmento siguiente muestra la implementación de la protección CSRF.

La implementación de la protección CSRF refleja los requisitos de seguridad del análisis. Las peticiones seguras —GET, HEAD y OPTIONS— se eximen de la comprobación y establecen la cookie del token si no existe; el endpoint de inicio de sesión se exige porque se limita en frecuencia de forma independiente. Las peticiones que modifican el estado exigen que el token de la cookie coincida con el de la cabecera `x-csrf-token`, que la interfaz envía de forma automática en las peticiones asíncronas; si no coinciden, el middleware responde HTTP 403 sin procesar la petición. Las cabeceras de seguridad se añaden en todas las respuestas, y el limitador de peticiones aplica un límite global de sesenta peticiones por minuto y un límite específico de cinco peticiones por minuto al endpoint de inicio de sesión, en coherencia con las pruebas de protección del capítulo 17.

El backend de vitalXAI queda así codificado de forma coherente con el diseño: la aplicación se compone en el arranque con los mecanismos de seguridad y los routers de los seis subsistemas, la capa HTTP sigue el patrón de fachada ligera, los servicios concentran la lógica de aplicación, la persistencia se resuelve con el pool de conexiones y la inicialización del esquema, y la seguridad transversal protege las peticiones y las respuestas de toda la aplicación. La implementación de la ejecución asíncrona, que constituye el soporte del procesamiento en segundo plano, se describe en el capítulo siguiente.

```

class CSRFMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next: RequestResponseEndpoint) -> Response:
        if request.method in SAFE_METHODS or request.url.path in CSRF_EXEMPT_PATHS:
            response = await call_next(request)
            if CSRF_COOKIE_NAME not in request.cookies:
                token = secrets.token_urlsafe(32)
                response.set_cookie(key=CSRF_COOKIE_NAME, value=token, httponly=False, samesite="lax")
            return response

        csrf_cookie = request.cookies.get(CSRF_COOKIE_NAME)
        csrf_token = request.headers.get(CSRF_HEADER_NAME)
        if not csrf_cookie or not csrf_token or csrf_cookie != csrf_token:
            return JSONResponse(status_code=403, content={"status": "error", "message": "CSRF validation failed"})

        response = await call_next(request)
        return response

```

Código 5 - Protección CSRF (services/csrf_middleware.py)

3. Codificación de la ejecución asíncrona

La ejecución asíncrona constituye el soporte del procesamiento en segundo plano de vitalXAI: los diagnósticos, los entrenamientos y las validaciones externas se procesan fuera del ciclo de petición HTTP, de modo que la interfaz permanece operativa durante las tareas de larga duración. Este capítulo describe la codificación de este mecanismo, que se materializa en la cola persistente de trabajos y en el worker de la cola. La implementación se organiza en cinco apartados: la cola de trabajos y su máquina de estados, el bucle del worker, la reclamación y el despacho de los trabajos, la tolerancia a fallos y la recuperación, y la consulta y la cancelación desde la API.

La implementación de la ejecución asíncrona se apoya en la persistencia relacional descrita en el capítulo 29: la cola se materializa en la tabla `job_queue` de MySQL, que conserva los trabajos con su estado, su payload serializable, su resultado y su error, y el worker comparte esa persistencia con los routers, pero no comparte el ciclo de petición. Esta frontera, ya declarada en el diseño del subsistema SD-006, se implementa en el módulo `services/queue_worker.py`, que contiene el bucle del worker y las operaciones de transición de estado, y en el router `routers/queue.py`, que expone la consulta del estado y la cancelación de los trabajos.

3.1. *La cola de trabajos y su máquina de estados*

La cola de trabajos se implementa en la tabla `job_queue`, cuyas filas representan los trabajos asíncronos del sistema. Cada trabajo conserva el usuario propietario, el tipo (diagnosis, training o external_validation), el estado, el payload serializable, el resultado y el mensaje de error, de modo que la tabla materializa la clase `QueueJob` definida en el diseño de clases del capítulo 22 y su máquina de estados, representada en la figura 89. Las transiciones de estado se implementan en el worker mediante operaciones que actualizan la fila del trabajo: la reclamación, la finalización y el fallo. El fragmento siguiente muestra la implementación de estas transiciones.

La implementación de las transiciones refleja la máquina de estados de la cola. La reclamación utiliza una actualización condicionada a que el trabajo continúe en `queued`, de modo que solo afecta a una fila si el trabajo seguía pendiente; el resultado de la actualización determina si el worker ha adquirido el trabajo, lo que impide que dos iteraciones procesen el mismo registro. La finalización marca el trabajo como `completed` con el resultado en formato JSON, y el fallo lo marca como `failed` con el mensaje de error limitado a quinientos caracteres, de modo que el estado persistido nunca conserva un error desbordado. La cancelación desde la API se implementa con la misma técnica condicionada, limitada a los trabajos en `queued`, tal y como se describe en el apartado 30.5.


```

def _claim_job(job_id: int):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE job_queue SET status = 'running', started_at = %s WHERE id = %s AND status = 'queued'",
        (datetime.now(), job_id),
    )
    conn.commit()
    affected = cursor.rowcount
    conn.close()
    return affected > 0

def _finish_job(job_id: int, result: dict | None = None):
    conn = get_db_connection()
    cursor = conn.cursor()
    result_json = json.dumps(result) if result else None
    cursor.execute(
        "UPDATE job_queue SET status = 'completed', finished_at = %s, result = %s WHERE id = %s",
        (datetime.now(), result_json, job_id),
    )
    conn.commit()
    conn.close()

def _fail_job(job_id: int, error: str):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE job_queue SET status = 'failed', finished_at = %s, error_message = %s WHERE id = %s",
        (datetime.now(), error[:500], job_id),
    )
    conn.commit()
    conn.close()

```

Código 6 - Transiciones de estado del trabajo (services/queue_worker.py)

3.2. *El bucle del worker*

El worker se implementa como un bucle asíncrono que selecciona el siguiente trabajo pendiente, lo reclama, procesa el flujo correspondiente y actualiza su estado, apoyado en el modelo de concurrencia asíncrona de Python (Python Software Foundation, 2024). El bucle se crea en el arranque de la aplicación mediante `start_worker()`, que restablece los trabajos en ejecución y lanza la tarea asíncrona del procesamiento. El fragmento siguiente muestra la implementación del bucle del worker.

La implementación del bucle refleja las decisiones de diseño del procesamiento asíncrono. El worker selecciona el primer trabajo pendiente mediante la consulta de prioridad, que ordena los diagnósticos antes que los entrenamientos; si no hay trabajos, espera un segundo y vuelve a consultar. Cuando hay un trabajo, lo reclama con la actualización condicionada; si la reclamación no afecta a una fila, otro consumidor ya lo tomó y el worker continúa con el siguiente. La ejecución del flujo se delega en el executor de eventos mediante `run_in_executor`, de modo que el procesamiento intensivo no bloquea el bucle de eventos de la aplicación; al terminar, el trabajo se marca como completado con el resultado, y ante cualquier excepción se marca como fallido con el mensaje de error. El arranque del worker se integra en el ciclo de vida de la aplicación descrito en el capítulo 29, y conserva la tarea en el estado de la aplicación.

```

async def worker_loop():
    loop = asyncio.get_running_loop()
    while True:
        try:
            job = _next_job()
            if job is None:
                await asyncio.sleep(1)
                continue

            claimed = _claim_job(job["id"])
            if not claimed:
                continue

            try:
                result = await loop.run_in_executor(None, _execute_job, job)
                _finish_job(job["id"], result)
            except Exception as e:
                _fail_job(job["id"], str(e))
        except Exception:
            await asyncio.sleep(1)

def start_worker(app):
    _reset_running_jobs()
    task = asyncio.create_task(worker_loop())
    app.state.queue_worker = task

```

Código 7 - Bucle del worker y su arranque (services/queue_worker.py)

3.3. *Reclamación, despacho y procesamiento de los trabajos*

El procesamiento de cada trabajo se resuelve mediante el despacho por tipo, que selecciona el flujo correspondiente según el tipo del trabajo. La reclamación condicionada, descrita en el apartado 30.1, garantiza que el despacho solo se ejecuta sobre trabajos que el worker ha adquirido. El fragmento siguiente muestra la implementación del despacho y del procesamiento de un diagnóstico.

La implementación del procesamiento de un diagnóstico refleja el flujo asíncrono definido en el diseño del subsistema SD-002. El worker deserializa el payload del trabajo, invoca el motor de predicción, genera el mapa de explicabilidad y construye el informe PDF, y persiste la consulta en la tabla consultations con las rutas de los artefactos y el resultado. El procesamiento se ejecuta en el executor, de modo que la predicción y la generación de los artefactos no bloquean la interfaz. Los procesamientos de los entrenamientos y de las validaciones externas siguen el mismo patrón y delegan en el motor del laboratorio MLOps, que se describe en el capítulo 32. El resultado devuelto se conserva en el campo result del trabajo completado.

```

def _execute_job(job):
    if job["job_type"] == "diagnosis":
        return _process_diagnosis(job)
    elif job["job_type"] == "training":
        return _process_training(job)
    elif job["job_type"] == "external_validation":
        return _process_external_validation(job)
    else:
        raise ValueError(f"Unknown job type: {job['job_type']}")

def _process_diagnosis(job):
    payload = _get_payload(job)
    label, confidence = process_and_predict(payload["model_name"], payload["image_path"], lang=payload.get("lang", "es"))

    xai_path = os.path.join("static", "results", "xai_" + os.path.basename(payload["image_path"]))
    generate_xai_heatmap(payload["model_name"], payload["image_path"], xai_path, lang=payload.get("lang", "es"))

    pdf_path = generate_medical_report(payload["image_path"], xai_path, label, confidence, payload["model_name"], lang=payload.get("lang", "es"))

    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO consultations (user_id, model_name, original_image_path, xai_image_path, prediction_label, confidence_score, patient_name, pdf_path) VALUES (%s, %s, %s, %s, %s, %s, %s, %s)",
        (job["user_id"], payload["model_name"], payload["image_path"], xai_path, label, confidence, default_patient_name, pdf_path),
    )
    conn.commit()
    conn.close()
    return {"label": label, "confidence": confidence, "original_image": f"/{payload['image_path']}",
            "xai_image": f"/{xai_path}", "pdf_report": f"/{pdf_path}", "model_used": payload["model_name"]}

```

Código 8 - Despacho por tipo y procesamiento de un diagnóstico (services/queue_worker.py)

3.4. Tolerancia a fallos y recuperación

La ejecución asíncrona incorpora mecanismos de tolerancia a fallos y de recuperación que garantizan la coherencia del sistema ante errores y reinicios. Ante un fallo del procesamiento, el worker captura la excepción y marca el trabajo como fallido con el mensaje de error, de modo que un fallo del motor, de la imagen o del informe no se transforma en una consulta completada con datos ambiguos. Ante el reinicio de la aplicación, los trabajos que quedaron en estado running se restablecen a queued, de modo que el procesamiento interrumpido vuelve a la cola y puede ser retomado. El fragmento siguiente muestra la implementación del restablecimiento de los trabajos en ejecución.

La implementación del restablecimiento refleja la decisión de recuperación del diseño. Al arrancar la aplicación, `start_worker()` ejecuta `_reset_running_jobs()`, que devuelve a la cola los trabajos que quedaron en running cuando el proceso anterior se detuvo, con su fecha de inicio restablecida; la reclamación condicionada evita que esos trabajos se procesen dos veces, porque solo un consumidor puede adquirir cada trabajo. Esta técnica, junto con la captura de excepciones del bucle, garantiza que el estado persistido de la cola permanece coherente con la ejecución real, incluso ante fallos del proceso o de los componentes.

```
def _reset_running_jobs():  
    conn = get_db_connection()  
    cursor = conn.cursor()  
    cursor.execute("UPDATE job_queue SET status = 'queued', started_at = NULL WHERE status = 'running'")  
    conn.commit()  
    conn.close()
```

Código 9 - Restablecimiento de los trabajos en ejecución (services/queue_worker.py)

3.5. *La cola desde la API*

La consulta del estado y la cancelación de los trabajos se exponen en el router `routers/queue.py`, que materializa los casos de uso CU-034 y CU-035 del análisis. La consulta del estado devuelve los trabajos del usuario con su tipo, su estado, su posición y el nombre del modelo o de la sesión, interpretando el payload según el tipo; la cancelación aplica una actualización condicionada que solo afecta a los trabajos en `queued` pertenecientes al usuario. El fragmento siguiente muestra la implementación de la cancelación.

La implementación de la cancelación refleja la restricción del diseño: solo un trabajo en estado `queued` y perteneciente al usuario puede cancelarse. La actualización condicionada a esos tres criterios afecta a una fila únicamente cuando el trabajo sigue pendiente; si el worker ya lo reclamó o el trabajo finalizó, la actualización no afecta a ninguna fila y el router responde HTTP 404 informando de que el trabajo no está en cola. De este modo, un trabajo en ejecución no se interrumpe de forma abrupta, en coherencia con la máquina de estados de la cola. La consulta del estado, por su parte, interpreta el payload de cada tipo de trabajo para mostrar el nombre del modelo o de la sesión sin exponer el contenido interno completo del payload.

La ejecución asíncrona de `vitalXAI` queda así codificada de forma completa: la cola persistente materializa la máquina de estados de los trabajos, el worker reclama y procesa los trabajos fuera del ciclo de petición, el despacho por tipo orquesta los flujos de los subsistemas, los mecanismos de tolerancia a fallos y de recuperación mantienen la coherencia del estado, y la API expone la consulta y la cancelación a los usuarios. La implementación de los motores que el worker invoca —la predicción y la explicabilidad del diagnóstico y el laboratorio `MLOps`— se describe en los capítulos siguientes.


```

@router.delete("/api/queue/cancel/{job_id}")
async def cancel_job(request: Request, job_id: int):
    user_id = get_user_id_from_token(request.cookies.get("access_token"))
    if not user_id:
        return JSONResponse(status_code=401, content={"error": "No autenticado"})

    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE job_queue SET status = 'cancelled', finished_at = NOW() WHERE id = %s AND user_id = %s AND status = 'queued'",
        (job_id, user_id),
    )
    affected = cursor.rowcount
    conn.commit()
    conn.close()

    if affected == 0:
        return JSONResponse(status_code=404, content={"error": "Trabajo no encontrado o ya no está en cola"})
    return JSONResponse(content={"status": "success", "message": "Trabajo cancelado"})

```

Código 10 - Cancelación de un trabajo desde la API (routers/queue.py)

4. Codificación del motor de diagnóstico y XAI

El motor de diagnóstico y de explicabilidad constituye el núcleo clínico de vitalXAI: recibe una radiografía de tórax, produce la predicción de neumonía con su nivel de confianza y genera los mapas de calor que explican la decisión del modelo, además del informe PDF de la consulta. Este capítulo describe la codificación de este motor, que se materializa en tres servicios: `ml_engine.py`, que carga los modelos y ejecuta la predicción; `xai_generator.py`, que genera los mapas de explicabilidad; y `pdf_generator.py`, que construye el informe del diagnóstico. La implementación se organiza en cuatro apartados: la carga y la caché de los modelos, el preprocesado y la predicción, la generación de los mapas de explicabilidad y la generación del informe PDF.

El motor se apoya en las librerías de aprendizaje profundo y de visión por computador descritas en el entorno de construcción del capítulo 24: TensorFlow y la librería Transformers de Hugging Face para la carga de las arquitecturas, OpenCV para el preprocesado de las imágenes y FPDF para la generación del informe (TensorFlow, 2024; Hugging Face, 2024). La implementación distingue dos familias de arquitecturas —las convolucionales (CNN) y las Transformer—, cuyas decisiones de carga, preprocesado e inferencia se bifurcan en cada servicio, en coherencia con el diseño del subsistema SD-002 descrito en el capítulo 18.

4.1. *Carga y caché de los modelos*

La carga de los modelos se implementa en el servicio `ml_engine.py`, que diferencia el procedimiento según la familia de la arquitectura. Los modelos convolucionales se cargan desde sus pesos entrenados en formato Keras; los modelos Transformer se cargan desde su arquitectura preentrenada de Hugging Face y, a continuación, se aplican los pesos entrenados del proyecto. En ambos casos, el modelo se conserva en un diccionario global de caché, de modo que la primera consulta con cada arquitectura paga la carga de los pesos y las posteriores reutilizan el modelo en memoria. El fragmento siguiente muestra la implementación de la carga.

La implementación de la carga refleja las decisiones del diseño del motor. La diferenciación por familia permite cargar cada arquitectura con el procedimiento adecuado: las CNN se cargan directamente con el cargador de modelos de Keras, y los Transformer se construyen sobre la arquitectura de Hugging Face, con la salida de atención habilitada para la generación de los mapas de explicabilidad, y reciben los pesos entrenados del proyecto. La caché en memoria implementa el requisito de tiempo de respuesta de la inferencia (RNF-019): la primera consulta con cada arquitectura asume el coste de la carga, mientras que las posteriores reutilizan el modelo. La ausencia de los pesos se resuelve con un error explícito, de modo que un modelo no disponible falla antes de producir un resultado ambiguo, tal y como se declaró en el diseño.

```

loaded_models = {}
MODELS_TRANSFORMERS = ["deit", "swin_base", "vit_384"]
HF_MODEL_IDS = {
    "deit": "facebook/deit-base-distilled-patch16-224",
    "swin_base": "microsoft/swin-base-patch4-window7-224",
    "vit_384": "google/vit-base-patch16-384",
}

def get_model(model_name: str):
    if model_name not in loaded_models:
        is_transformer = model_name in MODELS_TRANSFORMERS

        if is_transformer:
            from transformers import TFAutoModelForImageClassification
            model_path = os.path.join("pneumoniaccn-main", "results", model_name, "best_fold1.h5")
            if not os.path.exists(model_path):
                model_path = os.path.join("pneumoniaccn-main", "results", model_name, "best_fold1.weights.h5")
            if not os.path.exists(model_path):
                raise FileNotFoundError(f"Pesos de Transformer no encontrados en: {model_path}")

            model = TFAutoModelForImageClassification.from_pretrained(
                HF_MODEL_IDS[model_name], num_labels=1, ignore_mismatched_sizes=True, output_attentions=True
            )
            model.load_weights(model_path)
            loaded_models[model_name] = model
        else:
            model_path = os.path.join("pneumoniaccn-main", "results", model_name, "best_fold1.keras")
            if not os.path.exists(model_path):
                raise FileNotFoundError(f"Modelo CNN no encontrado en: {model_path}")
            loaded_models[model_name] = load_model(model_path)

    return loaded_models[model_name]

```

Código 11 - Carga y caché de los modelos (services/ml_engine.py)

4.2. *Preprocesado y predicción*

La predicción se implementa en la función `process_and_predict`, que prepara la imagen según la arquitectura y ejecuta la inferencia. El preprocesado ajusta la resolución de entrada a la arquitectura seleccionada —299×299 para InceptionV3 y Xception, 384×384 para ViT-384 y 224×224 para el resto—, convierte la imagen al espacio RGB y la normaliza. La inferencia se bifurca por familia: los Transformer procesan la imagen con los `pixel_values` y aplican una sigmoide sobre el logit, mientras que las CNN utilizan la predicción directa del modelo. El fragmento siguiente muestra la implementación del preprocesado y de la inferencia.

La implementación de la predicción refleja las decisiones del diseño clínico. La normalización divide los píxeles entre 255 para escalarlos al intervalo de entrada de los modelos; la inferencia devuelve la probabilidad de la clase neumonía, y el umbral de 0.5 decide la etiqueta de la predicción. La confianza se expresa en el intervalo 0-100: para la clase neumonía coincide con la probabilidad, y para la clase normal se calcula como su complementaria, de modo que la confianza siempre refleja la seguridad de la predicción emitida. Las etiquetas se localizan mediante el servicio de idioma, en coherencia con la internacionalización de la plataforma.

```

def process_and_predict(model_name: str, image_path: str, lang: str = "es"):
    model = get_model(model_name)
    is_transformer = model_name in MODELS_TRANSFORMERS

    if model_name in ["InceptionV3", "Xception"]:
        img_size = (299, 299)
    elif model_name == "vit_384":
        img_size = (384, 384)
    else:
        img_size = (224, 224)

    img = cv2.imread(image_path)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img_resized = cv2.resize(img_rgb, img_size)
    img_array = img_resized / 255.0
    img_tensor = np.expand_dims(img_array, axis=0)

    if is_transformer:
        img_tensor_tf = tf.convert_to_tensor(img_tensor, dtype=tf.float32)
        logits = model(pixel_values=img_tensor_tf).logits
        prediction = float(tf.nn.sigmoid(logits)[0][0])
    else:
        prediction = float(model.predict(img_tensor, verbose=0)[0][0])

    is_pneumonia = prediction > 0.5
    label = get_text("label_pneumonia", lang) if is_pneumonia else get_text("label_normal", lang)
    confidence = prediction if is_pneumonia else (1 - prediction)
    confidence_percent = round(confidence * 100, 2)

    return label, confidence_percent

```

Código 12 - Preprocesado y predicción (services/ml_engine.py)

4.3. Generación de los mapas de explicabilidad

La generación de los mapas de explicabilidad se implementa en el servicio `xai_generator.py`, que calcula los tres mapas de la explicación visual y compone la figura con la radiografía original. Para las arquitecturas convolucionales se aplican los métodos basados en gradientes —Saliency, SmoothGrad y Grad-CAM—, mientras que para las Transformer se utiliza el mapa de atención de la última capa, con un respaldo basado en gradientes ante un fallo del cálculo de la atención. El fragmento siguiente muestra la implementación de la generación del mapa de explicabilidad.

La implementación de la explicabilidad refleja las decisiones del diseño del subsistema de diagnóstico. El mapa de Saliency se calcula mediante los gradientes de la puntuación de la clase respecto a la imagen, aplicados con una cinta de gradientes de TensorFlow; el mapa de SmoothGrad promedia treinta mapas de Saliency con ruido gaussiano y normaliza el resultado; y el mapa de Grad-CAM pondera las activaciones de la última capa convolucional por los gradientes, mientras que las Transformer producen un mapa de atención a partir de la última capa de atención. La figura resultante combina la radiografía original, el Saliency, el SmoothGrad y la superposición del mapa principal, se guarda con el backend no interactivo de matplotlib y se persiste su ruta en la consulta, de modo que las visualizaciones posteriores no repiten el cálculo.

```

def generate_xai_heatmap(model_name: str, original_image_path: str, xai_save_path: str, lang: str = "es"):
    model = get_model(model_name)
    is_transformer = model_name in MODELS_TRANSFORMERS
    img_size = get_img_size(model_name)
    xai_method_name = "Attention Map" if is_transformer else "Grad-CAM"

    img = load_img_tf(original_image_path, img_size)
    sal = saliency(model, img, is_transformer)
    sm = smoothgrad(model, img, is_transformer)
    cam = get_cam_or_attention(model, img, is_transformer, img_size)

    plt.figure(figsize=(15, 5))
    plt.subplot(1, 4, 1)
    plt.imshow(img)
    plt.title(get_text("xai_original", lang), fontsize=12)
    plt.axis("off")
    # ... subplots de Saliency, SmoothGrad y la superposición del Grad-CAM o la atención ...
    plt.tight_layout()
    plt.savefig(xai_save_path, bbox_inches="tight", dpi=150)
    plt.close()
    return xai_save_path

```

Código 13 - Generación del mapa de explicabilidad (services/xai_generator.py)

4.4. Generación del informe PDF

La generación del informe PDF se implementa en el servicio `pdf_generator.py`, que compone el documento de la consulta mediante la clase `PDFReport`, especialización de la librería `FPDF`. El informe incluye la fecha, el modelo utilizado, el diagnóstico con su color según el resultado, el nivel de confianza y las dos imágenes —la radiografía original y el mapa de explicabilidad—. El fragmento siguiente muestra la implementación de la generación del informe.

La implementación del informe refleja las decisiones del diseño del CU-037. El documento se compone con la clase `PDFReport`, que hereda de la librería y redefine la cabecera y el pie de página; el color del diagnóstico se determina según el resultado —rojo para la neumonía y verde para el resultado normal—, considerando las etiquetas en los idiomas soportados. Las imágenes se insertan en paralelo y, si la carga de alguna falla, se muestra un texto de error en su lugar, de modo que el documento no queda incompleto. El informe se guarda en el área de informes con un nombre basado en la fecha y se devuelve la ruta, que se persiste en la consulta para su descarga posterior.

El motor de diagnóstico y de explicabilidad de `vitalXAI` queda así codificado de forma completa: la carga y la caché de los modelos atienden la inferencia con las dos familias de arquitecturas, el preprocesado y la predicción producen la etiqueta y la confianza, la generación de los mapas de explicabilidad proporciona la justificación visual de la decisión y la generación del informe PDF materializa el documento de la consulta. Estos servicios, invocados por el worker de la cola descrito en el capítulo 30, constituyen el núcleo clínico de la plataforma; la codificación del laboratorio de experimentación `MLOps` se describe en el capítulo siguiente.


```

def generate_medical_report(image_path, xai_path, label, confidence, model_name, lang="es"):
    pdf = PDFReport(lang=lang)
    pdf.add_page()
    pdf.set_font("helvetica", "", 12)

    pdf.cell(0, 10, f" {get_text('pdf_date', lang)} {datetime.now().strftime('%Y-%m-%d %H:%M')}")
    pdf.cell(0, 10, f" {get_text('pdf_model', lang)} {model_name}")
    pdf.ln(10)

    pdf.set_font("helvetica", "B", 14)
    if "neumonía" in label.lower() or "pneumonia" in label.lower() or "肺炎" in label or "न्यूमोनिया" in label:
        pdf.set_text_color(192, 57, 43)
    else:
        pdf.set_text_color(39, 174, 96)

    pdf.cell(0, 10, get_text("pdf_diagnosis", lang).format(label=label.upper()))
    pdf.set_text_color(0)
    pdf.cell(0, 10, get_text("pdf_confidence", lang).format(confidence=confidence))
    pdf.ln(10)

    try:
        pdf.image(image_path, 20, 100, 70)
        pdf.image(xai_path, 110, 100, 70)
    except Exception:
        pdf.cell(0, 10, get_text("pdf_error_images", lang))

    filename = f"report_{datetime.now().strftime('%Y%m%d%H%M%S')}.pdf"
    filepath = os.path.join("static", "reports", filename)
    pdf.output(filepath, "F")
    return filepath

```

Código 14 - Generación del informe PDF (services/pdf_generator.py)

5. Codificación del laboratorio MLOps

El laboratorio MLOps constituye el segundo núcleo funcional de vitalXAI: permite configurar, lanzar y analizar experimentos de entrenamiento de modelos de detección de neumonía, con validación cruzada, análisis de explicabilidad, comparación estadística y validación externa sobre cohortes independientes. Este capítulo describe la codificación de este laboratorio, que se materializa en el servicio `mlops_engine.py`, que organiza las sesiones de experimentación y orquesta el pipeline, y en el servicio `pdf_generator_mlops.py`, que genera el informe consolidado de cada sesión. La implementación se organiza en cinco apartados: la orquestación del pipeline de entrenamiento, la validación externa y la comparación estadística, la gestión de las sesiones, la lectura de los resultados y la generación del informe de la sesión.

La codificación del laboratorio sigue la decisión de diseño del capítulo 24: el pipeline de experimentación se ejecuta mediante los scripts del directorio `pneumoniacnn-main/code`, que el motor invoca como procesos externos con la configuración de cada trabajo transmitida por variables de entorno, apoyándose en el módulo de ejecución de procesos de Python (Python Software Foundation, 2024). Los scripts realizan el entrenamiento con validación cruzada, el análisis XAI cualitativo y cuantitativo, la comparación estadística y la validación externa, y escriben sus resultados en el directorio `training_results` de la sesión. El motor orquesta la ejecución de los scripts y resuelve la lectura de los resultados, manteniendo la persistencia híbrida de la plataforma: la cola y el estado en MySQL, y los artefactos en el sistema de ficheros.

5.1. Orquestación del pipeline de entrenamiento

La orquestación del entrenamiento se implementa en la función `run_training_queue`, que ejecuta el pipeline completo de una sesión: por cada modelo configurado, lanza el script de entrenamiento adecuado —convolucional o Transformer— y los scripts de análisis XAI, y al finalizar ejecuta el script de comparación estadística. La configuración de cada trabajo se transmite a los scripts mediante variables de entorno, y el registro de la ejecución se escribe en el archivo de log del entrenamiento. El fragmento siguiente muestra la implementación del bucle de orquestación.

La implementación de la orquestación refleja la decisión de diseño de delegar la computación intensiva en los scripts del pipeline. El motor itera sobre los modelos de la sesión, transmite la configuración mediante las variables de entorno `TFG_*` y ejecuta el script de entrenamiento correspondiente a la familia —el script 1 para las CNN o el script 2 para las Transformer—, seguido de los scripts de análisis XAI cualitativo y cuantitativo. La salida de los scripts se dirige al archivo de registro, de modo que la consola del laboratorio refleja la progresión del entrenamiento. Al completar todos los modelos, se ejecuta el script de comparación estadística, que genera el ranking y la matriz de Wilcoxon. Este flujo se ejecuta en el worker de la cola descrito en el capítulo 30, fuera del ciclo de petición.

```

def run_training_queue(session_id, models, dataset_path, epochs, batch_size, learning_rate):
    base_path = os.getcwd()
    script_train_cnn = os.path.join(base_path, "pneumoniaccnn-main", "code", "1_train_kfold.py")
    script_train_trans = os.path.join(base_path, "pneumoniaccnn-main", "code", "2_train_transformer_kfold.py")
    script_img = os.path.join(base_path, "pneumoniaccnn-main", "code", "6_xai_qualitative.py")
    script_math = os.path.join(base_path, "pneumoniaccnn-main", "code", "7_xai_quantitative.py")

    for model_name in models:
        env_vars = os.environ.copy()
        env_vars.update({
            "TFG_SESSION_ID": session_id, "TFG_MODEL_NAME": model_name,
            "TFG_DATASET_DIR": dataset_path, "TFG_EPOCHS": str(epochs),
            "TFG_BATCH_SIZE": str(batch_size), "TFG_LEARNING_RATE": str(learning_rate),
        })
        is_trans = model_name in TRANSFORMER_MODELS
        script_to_run = script_train_trans if is_trans else script_train_cnn
        with open(LOG_FILE, "a", encoding="utf-8", errors="replace") as log:
            subprocess.Popen(["python", script_to_run], stdout=log, stderr=subprocess.STDOUT, env=env_vars, text=True, encoding="utf-8", errors="replace").wait()
            subprocess.Popen(["python", script_img], stdout=log, stderr=subprocess.STDOUT, env=env_vars, text=True, encoding="utf-8", errors="replace").wait()
            subprocess.Popen(["python", script_math], stdout=log, stderr=subprocess.STDOUT, env=env_vars, text=True, encoding="utf-8", errors="replace").wait()

    env_vars_comp = os.environ.copy()
    env_vars_comp["TFG_SESSION_ID"] = session_id
    script_comp = os.path.join(base_path, "pneumoniaccnn-main", "code", "3_evaluate_statistics.py")
    with open(LOG_FILE, "a", encoding="utf-8", errors="replace") as log:
        subprocess.Popen(["python", script_comp], stdout=log, stderr=subprocess.STDOUT, env=env_vars_comp, text=True, encoding="utf-8", errors="replace").wait()

```

Código 15 - Orquestación del pipeline de entrenamiento (services/ml_ops_engine.py)

5.2. Validación externa y comparación estadística

La validación externa y la comparación estadística se implementan en las funciones `run_external_validation` y `run_statistical_comparison`, que ejecutan los scripts correspondientes del pipeline. La validación externa evalúa los modelos de la sesión sobre el dataset independiente y aplica el test de DeLong; la comparación estadística regenera el ranking y la matriz de Wilcoxon, gestionando el marcador de estado del recálculo. El fragmento siguiente muestra la implementación de la validación externa.

La implementación de la validación externa refleja la separación entre la validación y la configuración original del experimento: la función recibe la sesión y la ruta del dataset externo, transmite la ruta mediante la variable de entorno `TFG_EXTERNAL_DATASET_DIR` y ejecuta los scripts de validación y del test estadístico de DeLong, que escriben las métricas, la curva ROC y la matriz de DeLong en el subdirectorio `external_validation` de la sesión. La comparación estadística, por su parte, ejecuta el script 3 con el identificador de la sesión, limpia el marcador de estado del recálculo antes de ejecutarlo y lo recrea al finalizar, de modo que la interfaz puede consultar si el recálculo está en curso o completado.

```
def run_external_validation(session_id, dataset_path):
    base_path = os.getcwd()
    env_vars = os.environ.copy()
    env_vars["TFG_SESSION_ID"] = session_id
    env_vars["TFG_EXTERNAL_DATASET_DIR"] = dataset_path
    script_val = os.path.join(base_path, "pneumoniaccnn-main", "code", "4_external_validation.py")
    script_delong = os.path.join(base_path, "pneumoniaccnn-main", "code", "5_evaluate_delong.py")
    with open(LOG_FILE, "a", encoding="utf-8", errors="replace") as log:
        subprocess.Popen(["python", script_val], stdout=log, stderr=subprocess.STDOUT, env=env_vars, text=True, encoding="utf-8", errors="replace").wait()
        subprocess.Popen(["python", script_delong], stdout=log, stderr=subprocess.STDOUT, env=env_vars, text=True, encoding="utf-8", errors="replace").wait()
```

Código 16 - Validación externa (services/ml_ops_engine.py)

5.3. *Gestión de las sesiones de experimentación*

La gestión de las sesiones se implementa en el motor MLOps, que organiza el directorio de cada sesión en `training_results` y resuelve las operaciones de creación, consulta, renombrado y eliminación. La creación de una sesión materializa el identificador, la configuración y el propietario; la consulta enumera las sesiones del usuario con sus modelos; el renombrado y la eliminación operan sobre el directorio con las comprobaciones de validez y de propiedad. El fragmento siguiente muestra la implementación de la creación de la sesión.

La implementación de la creación de la sesión refleja la persistencia híbrida del laboratorio: la sesión se materializa en un directorio de `training_results` con su configuración en `config.json` y la ruta del dataset en `dataset_path.txt`. El identificador se genera a partir de la fecha y la hora, y la configuración incorpora los modelos, los hiperparámetros y el identificador del usuario propietario, que la comprobación de propiedad de las sesiones utiliza para aislar los datos entre usuarios. Las operaciones de renombrado y de eliminación operan sobre el directorio mediante el sistema de ficheros, con la sanitización del nuevo nombre y las comprobaciones de existencia y de propiedad descritas en el diseño.

```
def create_training_session(model_names, dataset_path, epochs, batch_size, learning_rate, user_id=None):
    session_id = f"RUN_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}"
    session_dir = f"training_results/{session_id}"
    os.makedirs(session_dir, exist_ok=True)
    with open(os.path.join(session_dir, "dataset_path.txt"), "w", encoding="utf-8") as f:
        f.write(dataset_path)
    config = {"dataset_path": dataset_path, "epochs": epochs, "batch_size": batch_size,
             "learning_rate": learning_rate, "models": [m.strip() for m in model_names.split(",")]}
    if user_id is not None:
        config["user_id"] = user_id
    with open(os.path.join(session_dir, "config.json"), "w", encoding="utf-8") as f:
        json.dump(config, f)
    return session_id
```

Código 17 - Creación de la sesión de entrenamiento (services/ml_ops_engine.py)

5.4. *Lectura de los resultados*

La lectura de los resultados se implementa en las funciones de consulta del motor, que transforman los artefactos persistidos en el sistema de ficheros en estructuras para la interfaz. La consulta de los resultados de un modelo lee las métricas de validación cruzada, la calibración, las métricas XAI y las rutas de los artefactos visuales; la consulta del ranking lee el ranking por AUC y el heatmap de Wilcoxon; y la consulta de la validación externa lee las métricas, la curva ROC y la matriz de DeLong. El fragmento siguiente muestra la implementación de la lectura del ranking.

La implementación de la lectura de los resultados refleja la decisión de no ejecutar el modelo en las consultas: las funciones transforman los ficheros de resultados persistidos por el pipeline —los CSV de métricas y los artefactos visuales— en las estructuras que la interfaz presenta, devolviendo None cuando los resultados no existen para que el router informe con el código correspondiente. Esta decisión reduce el coste de acceso a los resultados y conserva la separación entre el cálculo y la visualización, en coherencia con el diseño del laboratorio.


```
def get_session_ranking_data(session_id):
    session_dir = f"training_results/{session_id}"
    csv_path = f"{session_dir}/session_ranking.csv"
    if not os.path.exists(csv_path):
        return None
    data = []
    with open(csv_path, encoding="utf-8") as f:
        for row in csv.DictReader(f):
            data.append(row)
    heatmap = f"/training_results/{session_id}/wilcoxon_heatmap.png"
    return {"ranking": data, "heatmap": heatmap, "config": read_config(session_dir)}
```

Código 18 - Lectura del ranking de la sesión (services/mlops_engine.py)

5.5. Generación del informe de la sesión

La generación del informe de la sesión se implementa en el servicio `pdf_generator_mlops.py`, que compone el documento consolidado mediante la clase `MedicalReport`, especialización de la librería `FPDF`. El informe recoge la configuración del experimento, el ranking con el heatmap de Wilcoxon, los resultados de la validación externa con la curva ROC y la matriz de DeLong, y el detalle técnico de cada modelo con sus métricas XAI y sus mapas de calor. El fragmento siguiente muestra el inicio de la implementación del generador.

La implementación del informe refleja la decisión de diseño de separar el conocimiento del formato del router: el generador recibe la sesión, lee los artefactos persistidos en el sistema de ficheros —la configuración, el ranking, los heatmaps, las métricas de la validación externa y las métricas XAI de cada modelo— y compone el documento con la clase `MedicalReport`, que redefine la cabecera corporativa, el pie de página y los títulos de sección. El informe se guarda en el directorio de la sesión y se devuelve como documento descargable; si la sesión no existe, el generador responde HTTP 404.

El laboratorio MLOps de vitalXAI queda así codificado de forma completa: el motor orquesta el pipeline de entrenamiento y de validación externa mediante los scripts del proyecto, gestiona las sesiones en el sistema de ficheros, lee los resultados persistidos para la interfaz y genera el informe consolidado de cada sesión. Con la codificación del motor de diagnóstico y del laboratorio, los motores del sistema quedan implementados; la codificación del frontend, que presenta estas capacidades al usuario, se describe en el capítulo siguiente.

```

async def generate_pdf_report(session_id):
    session_dir = f"training_results/{session_id}"
    if not os.path.exists(session_dir):
        return JsonResponse(status_code=404, content={"message": "Sesión no encontrada"})

    pdf = MedicalReport()
    pdf.set_auto_page_break(auto=True, margin=20)
    pdf.add_page()
    pdf.section_title("1. CONFIGURACIÓN DEL SISTEMA Y PARÁMETROS")

    config_path = os.path.join(session_dir, "config.json")
    if os.path.exists(config_path):
        with open(config_path, encoding="utf-8") as f:
            cfg = json.load(f)
            # ... secciones de configuración, ranking, Wilcoxon, validación externa y detalle por modelo ...

    pdf_output_path = os.path.join(session_dir, f"Informe_Completo_{session_id}.pdf")
    pdf.output(pdf_output_path)
    return FileResponse(pdf_output_path, filename=f"Reporte_MLOps_{session_id}.pdf")

```

Código 19 - Generación del informe de la sesión (services/pdf_generator_mlops.py)

6. Codificación del frontend

El frontend de vitalXAI materializa la presentación del sistema: las ventanas descritas en el diseño de interfaces del capítulo 23, que permiten al profesional sanitario y al investigador interactuar con las capacidades del backend. El frontend se construye con las plantillas Jinja2 servidas por el backend y con los recursos JavaScript del navegador, sin un framework de componentes independiente, en coherencia con la decisión tecnológica adoptada en el análisis. Este capítulo describe la codificación del frontend, organizada en cuatro apartados: las plantillas de presentación, los recursos JavaScript por ámbito, la comunicación asíncrona con la API y la internacionalización con el tema visual.

La presentación se resuelve con las plantillas Jinja2 del directorio `templates/`, que componen las páginas en el servidor, y con los recursos estáticos del directorio `static/`, que incluyen los scripts JavaScript y los estilos. La interactividad se implementa en JavaScript nativo, con módulos por ámbito funcional, y la comunicación con la API se realiza mediante la interfaz de `fetch` del navegador, con la protección CSRF y la renovación automática de la sesión (MDN Web Docs, 2024). El estilo visual se apoya en la utilidades de Tailwind CSS, cargadas mediante CDN y configuradas con el modo oscuro mediante clase (Tailwind CSS, 2024).

6.1. *Las plantillas de presentación*

Las plantillas Jinja2 componen las páginas del sistema en el servidor, inyectando los datos del usuario y los textos de la interfaz. Las páginas principales son el inicio de sesión (`login.html`), el registro (`register.html`), el panel de diagnóstico (`dashboard.html`) y el laboratorio de entrenamiento (`training.html`). Las plantillas combinan la estructura HTML con los atributos de internacionalización `data-i18n`, que el script de idioma traduce dinámicamente, y con las variables de contexto del servidor, como el nombre y el rol del usuario. El fragmento siguiente muestra una sección de la plantilla del panel de diagnóstico.

La implementación de las plantillas refleja la decisión del diseño de interfaces: las páginas se componen en el servidor con Jinja2, que inyecta el nombre y el rol del usuario, y se sirven como respuestas HTML. Las secciones de la interfaz se identifican con atributos `data-i18n`, que el módulo de idioma traduce sin recargar la página, y la navegación entre el panel y el laboratorio se resuelve con los enlaces de la barra lateral. Las ventanas integran además las vistas del historial, la cola de trabajos y la administración, tal y como se describió en el diseño de interfaces del capítulo 23.

```

<div class="p-4 border-b ... bg-blue-50 ...">
  <div class="flex items-center gap-3">
    <div class="w-10 h-10 rounded-full bg-blue-600 ...">
      <i class="fa-solid fa-user-md text-lg"></i>
    </div>
    <div class="overflow-hidden">
      <p class="text-[10px] font-bold ...">{{ role }}</p>
      <p class="text-sm font-black ...">{{ full_name }}</p>
    </div>
  </div>
</div>

<div class="p-3 space-y-1 border-b ...">
  <a href="/dashboard" class="flex items-center gap-3 px-3 py-2 rounded-lg bg-blue-100 ...">
    <i class="fa-solid fa-stethoscope w-5 text-center"></i>
    <span id="ui-nav-diag" data-i18n="navDiag">Diagnóstico Rápido</span>
  </a>
  <a href="/training" class="flex items-center gap-3 px-3 py-2 rounded-lg ...">
    <i class="fa-solid fa-flask w-5 text-center"></i>
    <span id="ui-nav-lab" data-i18n="navLab">Laboratorio de Entrenamiento</span>
  </a>
</div>

```

Código 20 - Plantilla del panel de diagnóstico (templates/dashboard.html)

6.2. Los recursos JavaScript por ámbito

La interactividad del frontend se implementa en JavaScript nativo, organizado en módulos por ámbito funcional: `dashboard.js` para el panel de diagnóstico, `training.js` para el laboratorio, `admin.js` para la administración y la cola, e `i18n.js` para la internacionalización compartida. Cada módulo se encarga de la interacción de su ventana: la carga y el envío de la radiografía, el sondeo del estado de la cola, el render de los resultados, la gestión de los experimentos del laboratorio y los diálogos de la administración. Los módulos se cargan en las plantillas al final del documento, y se comunican con la API mediante las peticiones asíncronas descritas en el apartado siguiente.

La separación por ámbito refleja la organización del código del frontend: cada módulo conoce los elementos de su ventana y las operaciones de su subsistema, de modo que los cambios de un ámbito no afectan al resto. El módulo `i18n.js` es el único compartido, y se carga en todas las páginas para aplicar el idioma y el tema visual. La coordinación entre módulos se resuelve mediante la ventana global, que expone las funciones de interacción que las plantillas invocan desde los atributos de los elementos, como el envío de la imagen o la apertura del detalle de una consulta.

6.3. Comunicación asíncrona con la API

La comunicación con la API se realiza mediante la interfaz de `fetch` del navegador, con el envío de los formularios de forma asíncrona mediante `FormData`, de modo que las operaciones se procesan sin recargar la página. La protección CSRF se gestiona mediante un interceptor global de peticiones, que añade la cabecera `X-CSRF-Token` a las peticiones que modifican el estado y gestiona la renovación automática de la sesión ante una respuesta de no autenticado. El fragmento siguiente muestra la implementación del interceptor global.

La implementación de la comunicación asíncrona refleja las decisiones de seguridad y de experiencia del sistema. El interceptor reemplaza la función `fetch` global: añade la cabecera `X-CSRF-Token` con el token de la cookie a todas las peticiones que modifican el estado, en correspondencia con la protección CSRF del backend descrita en el capítulo 29, y ante una respuesta de no autenticado intenta renovar la sesión mediante el endpoint de refresco, reintentando la petición original si la renovación tiene éxito o redirigiendo al inicio de sesión en caso contrario. Este mecanismo, aplicado de forma transversal, mantiene la sesión del usuario sin interrupciones y evita duplicar la gestión de la autenticación en cada módulo.

```

(function() {
  const c = document.cookie.match(/csrf_token=([^\;]+)/);
  if (!c) return;
  const csrfToken = c[1];
  const origFetch = window.fetch;
  let _isRefreshing = false;
  window.fetch = async function(url, options) {
    if (options && (options.method || 'GET').toUpperCase() !== 'GET') {
      options = Object.assign({}, options);
      options.headers = Object.assign({}, options.headers || {});
      options.headers['X-CSRF-Token'] = csrfToken;
    }
    let response = await origFetch(url, options);
    if (response.status === 401 && !_isRefreshing) {
      _isRefreshing = true;
      try {
        const refreshResp = await origFetch('/api/token/refresh', { method: 'POST', headers: { 'X-CSRF-Token': csrfToken } });
        if (refreshResp.ok) {
          response = await origFetch(url, options);
        } else {
          window.location.href = '/';
          return response;
        }
      } catch (e) {
        window.location.href = '/';
      }
    }
    return response;
  };
})();

```

Código 21 - Interceptor global de peticiones con CSRF y renovación de sesión (static/js/admin.js)

6.4. Internacionalización y tema visual

La internacionalización y el tema visual se implementan en el módulo `i18n.js`, compartido por todas las páginas. La internacionalización mantiene un diccionario de traducciones en el navegador para los cuatro idiomas soportados, aplica los textos a los elementos `data-i18n` y persiste la preferencia del idioma en el almacenamiento local; el tema visual alterna entre el tema claro y el oscuro aplicando la clase correspondiente en el elemento raíz y respeta la preferencia del sistema cuando no hay una guardada. El fragmento siguiente muestra la implementación de la traducción y del cambio de idioma.

La implementación de la internacionalización refleja las decisiones del diseño del CU-004: el selector de idioma actualiza la preferencia en el almacenamiento local y traduce de forma inmediata los textos de la interfaz marcados con `data-i18n`, sin recargar la página, con el español como valor por defecto. El tema visual alterna la clase del modo oscuro en el elemento raíz, de modo que el cambio se propaga a toda la interfaz, y persiste la preferencia en el almacenamiento local; al cargar la página, se restaura la preferencia guardada y, en ausencia de esta, se respeta la preferencia de color del sistema. Los módulos de las ventanas complementan la traducción re-renderizando las vistas dinámicas, como el historial o los resultados del laboratorio, en el idioma seleccionado.

El frontend de vitalXAI queda así codificado de forma completa: las plantillas Jinja2 componen las ventanas en el servidor, los recursos JavaScript por ámbito gestionan la interacción de cada vista, la comunicación asíncrona se protege con el interceptor CSRF y la renovación de sesión, y la internacionalización con el tema visual se aplican de forma transversal. Con la codificación del backend, de la ejecución asíncrona, de los motores y del frontend, la Parte de Codificación de la memoria queda completada, describiendo la implementación integral del sistema vitalXAI.


```
function t(key) {
  const lang = currentLang || localStorage.getItem('apLang') || 'es';
  const langDict = dict[lang] || dict.es;
  return langDict[key] || dict.es[key] || key;
}

function changeLanguage() {
  const selector = document.getElementById('lang-selector');
  if (selector) currentLang = selector.value;
  localStorage.setItem('apLang', currentLang);
  document.querySelectorAll('[data-i18n]').forEach(el => {
    const key = el.getAttribute('data-i18n');
    el.innerText = t(key);
  });
}

function toggleTheme() {
  const html = document.documentElement;
  if (html.classList.contains('dark')) {
    html.classList.remove('dark');
    localStorage.setItem('theme', 'light');
  } else {
    html.classList.add('dark');
    localStorage.setItem('theme', 'dark');
  }
}
```

Código 22 - Internacionalización y tema visual (static/js/i18n.js)